



Full Duplex

边听 / 边想 / 边说 / 可打断

Voice Identity

call / turn / generation / clip
claim

Cove Core

当前模型 / 工具能力 / 记忆系统 /
复刻音色

COVE REALTIME VOICE / IMPLEMENTATION GUIDE

Cove GPT-Live 式双工语音交互系统实战教程

从半双工语音助手升级到可打断、低延迟、能连续聊，并保留 Cove 自有模型、工具、记忆与复刻音色的自然对话链路。

— 目录

01 1. 先把“复刻 GPT Live”说准确

03 3. 延迟预算：不要只看总耗时

05 5. 会话状态机

07 7. 自适应停句：别用固定两秒静音

09 9. ASR：远端速度与本地质量并行竞速

11 11. 模型层：保留当前模型，但给语音一套轻上下文

13 13. 本地短反馈：填空档，但不能抢真实回答

15 15. 回声控制：避免把 AI 自己当成用户

17 17. 一轮完整对话的真实时序

19 19. 一组可落地的事件协议

21 21. 分阶段实现，不要一次把所有异步都打开

23 23. 我们踩过的坑，以及它们真正说明了什么

25 25. 一个最小可用版和一个完整升级版

27 27. 结语

02 2. 最终架构

04 4. 第一根支柱：给每个异步事件发身份证

06 6. 音频采集：一条媒体流，三种用途

08 8. 唤醒词：不要让通用 ASR 同时做人声门和关键词门

10 10. 语气感知不要阻塞正式回答

12 12. TTS：首句优先切分 + 持久连接 + 有序播放

14 14. 真正的核心：自然抢话（barge-in）

16 16. 自然挂断

18 18. 推荐项目结构

20 20. 环境配置示例

22 22. 测试：这类系统必须测“时序”，不只是测返回值

24 24. 可观测性与隐私

26 26. 上线前检查表

本文写给已经做过普通 ASR → LLM → TTS 语音链路，想继续把它升级成自然实时交互的开发者。

示例以 Python 异步后端、浏览器/Electron 前端、文本大模型、远端 ASR、本地 Whisper 和流式 TTS 为基础。代码是脱敏后的通用写法，参数来自一套经过真实麦克风、扬声器和多轮对话调试的生产系统。

1. 先把“复刻 GPT Live”说准确

我们最初也有一个很常见的语音助手：

用户说话

- 停止录音
- 上传整段音频
- ASR 转成文字
- 文本模型生成完整答案
- TTS 合成完整音频
- 播放

它能用，但不像通话。用户每次说完都要面对一段沉默；机器说话时用户不能自然插话；第二轮还可能播出第一轮迟到的音频。只要 ASR、模型、TTS 中任何一段抖动，整轮体验就像卡住。

后来我们把它升级成了一套“GPT Live 式”的自然语音系统。这里的“式”很重要：我们复刻的是交互体验和时序，不是照搬某个厂商的 WebRTC 音频到音频协议。

最终系统仍然允许你自由组合：

- 任意支持流式输出的聊天模型；
- 自己现有的工具调用、记忆和人格系统；
- 自己训练或购买的复刻音色；
- 远端快速 ASR 与本地高准确 ASR；
- Web、Electron、iPhone 等不同客户端。

它能做到：

- 接通后 AI 可以先自然开口；
- 用户说话时偶尔有轻声监听回应；
- 用户停句后立刻得到内容相关的短反馈；
- 模型一产生真实首句，马上开始合成和播放；
- AI 说话期间麦克风仍在工作，用户可以随时抢话；
- 抢话不会吞掉开头第一个字；
- 旧回复、旧音频和迟到的异步任务不会串进下一轮；
- “我先挂了”会先完成告别，再结束通话；
- 语音聊天继续使用主聊当前选择的模型和工具能力。

这不是靠某一个“实时 API”完成的。真正决定体验的是下面几件事：

- 1. 每条链路都流式或并行运行；
- 2. 所有事件都有明确身份；
- 3. 播放权可撤销，而且撤销必须彻底；
- 4. 本地反馈只填空档，真实回答永远优先；
- 5. 超时和失败被限制在局部，不能拖死整轮。

2. 最终架构



从图上看，系统并没有一条从头跑到尾的“大流水线”。它更像几条同时工作的轨道：

- 识别轨负责判断用户说了什么；
- 对话轨负责让模型尽快吐出可说的真实文本；
- 合成轨边收文本边生成音频；
- 播放轨决定现在谁能出声；
- 监听轨即使在播放时也继续看用户是否开口；
- 身份协议负责阻止旧任务污染新一轮。

只有把这些轨道分开，系统才真的能“边听、边想、边说、边被打断”。

3. 延迟预算：不要只看总耗时

普通语音系统经常只记录一个“请求耗时”，这对优化几乎没用。我们把一轮拆成至少六段：

阶段	起点	终点	推荐目标
endpoint	用户最后一个有效音节	客户端确认停句	0.6 – 1.35 秒
upload	停句确认	服务端收到可识别音频	0.05 – 0.3 秒
transcription	服务端收到音频	得到最终文字	热路径 0.5 – 2 秒
context	得到文字	模型请求发出	0.05 – 0.3 秒
model first text	模型请求发出	第一段可见文字	取决于模型，尽量小于 4 秒
TTS first audio	第一段文字送入 TTS	第一段音频可播放	0.3 – 1.2 秒

总等待可写成：

用户感知等待 $\approx$ endpoint + upload + ASR + context + model first text + TTS first audio - 被本地反馈覆盖的时间
--

这条公式提醒我们两件事。

第一，边录边识别不是万能药。如果生产日志显示主要瓶颈在模型首字，那么换成增量 ASR 也不会让正式回答突然快五秒。

第二，本地短反馈能改善“体感空白”，但它不能掩盖无限等待。每个远端调用仍要有明确超时，迟到任务仍要被取消。

建议每轮输出一条结构化日志：

```
{
  "event": "voice_latency",
  "call_id": "call_xxx",
  "turn_id": "turn_xxx",
  "generation_id": "gen_xxx",
  "endpoint_ms": 910,
  "transcription_ms": 820,
  "context_ms": 74,
  "model_first_text_ms": 2860,
  "tts_first_audio_ms": 610,
  "first_sound_ms": 1180,
  "formal_first_sound_ms": 5274,
  "asr_backend": "local-high-accuracy",
  "tts_transport": "websocket"
}
```

`first_sound_ms` 和 `formal_first_sound_ms` 要分开。前者可能是本地“好，我看看”，后者才是模型的真实回答。

4. 第一根支柱：给每个异步事件发身份证

自然双工系统最难处理的不是音频，而是“迟到”。

想象这个时序：

1. 用户问 A；
2. 模型 A 开始生成；
3. 用户打断并问 B；
4. 模型 A 的第二段 TTS 此时才返回；
5. 如果只看 WebSocket 是否还连接，A 的音频会插进 B。

所以不能只用一个 `isSpeaking` 布尔值。我们使用四层身份：

<code>call_session_id</code>	一通连续语音会话
<code>turn_id</code>	用户的一次发言
<code>turn_sequence</code>	会话内单调递增轮次
<code>generation_id</code>	对这一轮回答的某次生成

一个事件信封可以这样定义：

```
{
  "protocol_version": 1,
  "call_session_id": "call_01",
  "turn_id": "turn_07",
  "turn_sequence": 7,
  "generation_id": "gen_07_a",
  "seq": 0,
  "event": "tts_chunk",
  "text": "我先看看这个问题。",
  "audio_url": "/audio/abc.mp3"
}
```

客户端接收任何异步结果前，先做身份匹配：

```
function belongsToActiveGeneration(event, active) {
  return event.call_session_id === active.callId
    && event.turn_id === active.turnId
    && event.turn_sequence === active.turnSequence
    && event.generation_id === active.generationId;
}
```

这条检查要放在所有异步边界上：

- `fetch()` 返回后；
- WebSocket 消息到达时；
- `audio.play()` 的 Promise resolve/reject 时；
- 本地短音频播放完成时；
- TTS 合成结束时；
- 模型流结束时；
- `setTimeout()` 回调触发时；
- 自动重试准备执行时。

一个容易忽略的细节是：旧 Promise 的拒绝也可能污染新状态。例如上一轮 `audio.play()` 失败后执行 `currentAudio = null`，但此时 `currentAudio` 已经是下一轮的对象。解决办法不是多加一个布尔值，而是让播放对象也带不可复用的 claim：

```
const claim = {
  callId,
  turnId,
  generationId,
  clipId,
  nonce: crypto.randomUUID()
};

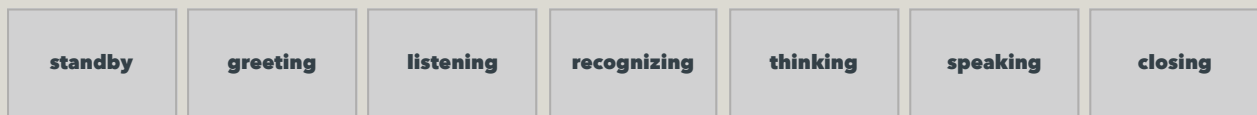
try {
  await audio.play();
  if (!sameClaim(claim, activeClaim)) return;
  commitPlayback(claim);
} catch (error) {
  if (sameClaim(claim, activeClaim)) {
    releasePlayback(claim);
  }
}
```

身份协议是后面所有“自然”的前提。没有它，越流式、越并行，串台越严重。

5. 会话状态机

一通电话至少需要这些状态：

一通 **Cove** 语音会话的状态机



用户抢话可以从 greeting / thinking / speaking 回到 listening；挂断必须先完成告别，再释放资源。

不要让 UI 自己猜状态。后端广播的状态至少包含：

```
{
  "mode": "listening",
  "call_session_id": "call_01",
  "turn_id": "turn_08",
  "generation_id": null,
  "capture_active": true,
  "speech_active": false,
  "message": "我在听。"
}
```


前端所有入口都调用同一个渲染函数，例如：

```
function applyVoiceStatus(status) {  
  renderHeaderIndicator(status);  
  renderSettingsStatus(status);  
  renderDesktopPetBubble(status);  
}
```

不要让“开关请求完成”“WebSocket 推送”“页面初始化”“定时状态轮询”分别修改不同的 DOM。我们真实遇到过这样的竞态：WebSocket 已经正确显示“监听中”，稍晚返回的 HTTP 开关请求又把文字覆盖成“正在校准”。后端没有坏，麦克风也没坏，只是两个前端写入者互相踩。

解决方法是单一渲染入口，再用低频状态轮询修复断线或延迟握手造成的漂移。轮询只用于对账，不参与实时语音数据传输。

6. 音频采集：一条媒体流，三种用途

浏览器端拿到麦克风：

```
const stream = await navigator.mediaDevices.getUserMedia({  
  audio: {  
    channelCount: 1,  
    sampleRate: 48000,  
    echoCancellation: false,  
    noiseSuppression: false,  
    autoGainControl: false  
  }  
});
```

这不是固定答案。通话播放期间为了降低回声污染，可以临时使用 `echoCancellation: true`；待机唤醒也可以保留降噪与自动增益，提高远场稳定性。关键是不要假设所有阶段都该共用同一套 DSP。

我们把同一条媒体流分成三种用途：

6.1 分析流

分析流提供给：

- VAD；
- adaptive endpoint；
- 抢话检测；
- 唤醒词声学门禁；
- 最终 ASR。

它需要低延迟、稳定的 PCM 帧。推荐使用 `AudioWorkletProcessor`，不要把生产级连续采集长期放在 `ScriptProcessorNode` 上。

```
// pcm-worklet.js
class PcmTapProcessor extends AudioWorkletProcessor {
  process(inputs) {
    const input = inputs[0]?.[0];
    if (!input) return true;

    const copy = new Float32Array(input.length);
    copy.set(input);
    this.port.postMessage(copy, [copy.buffer]);
    return true;
  }
}

registerProcessor("pcm-tap", PcmTapProcessor);
```

主线程接收后再分帧：

```
worklet.port.onmessage = ({ data }) => {
  endpointDetector.push(data, audioContext.sampleRate);
  bargeInDetector.push(data, audioContext.sampleRate);
  turnRecorder.push(data);
};
```

为什么不用主线程旧接口？我们曾遇到页面同时做动画、TTS 播放和状态更新时，主线程回调漏帧。程序把剩余 PCM 首尾拼接后，WAV 头和采样率都合法，但中间真实时间消失了。回放听起来像被加速，ASR 也更容易错。

6.2 识别流

识别流通常转换为 16kHz、单声道、16-bit PCM WAV：

```
48kHz Float32 PCM
→ 重采样到 16kHz
→ 限幅
→ Int16
→ WAV
```

这条流追求 ASR 稳定，不追求回放音质。

6.3 原声归档流

聊天里回放用户自己的语音，不能直接复用识别流。16kHz、强降噪和自动增益会让声音变薄、变挤，甚至产生“语速快了”的错觉。

更稳的做法是在连续对话期间并行启动浏览器原生 `MediaRecorder`：

```
const recorder = new MediaRecorder(stream, {
  mimeType: "audio/webm;codecs=opus"
});

const chunks = [];
recorder.ondataavailable = event => {
  if (event.data.size > 0) chunks.push(event.data);
};
```

每轮结束后：

1. 停止该轮对应的 recorder；
2. 解码带时间戳的原生录音；
3. 只裁掉首尾静音；
4. 保留中间停顿和原始时序；
5. 生成最高 48kHz 的回放附件；
6. 如果解码失败，再回退到 PCM 副本。

分析 PCM 与回放归档必须是两条逻辑管道。一个用于机器听懂，一个用于人类回听。

7. 自适应静音：别用固定两秒静音

固定静音阈值有两个明显问题：

- 阈值短，用户思考时容易被抢断；
- 阈值长，短句说完后等待很拖沓。

一个实用的起点是：

- 短句：约 900ms 静音收口；
- 较长发言：约 1350ms；
- 单句话硬上限：60 秒；
- 硬上限从真实起音开始算，不从“进入 listening”开始算。

简化实现：

```

class AdaptiveEndpointDetector {
  constructor() {
    this.speechMs = 0;
    this.silenceMs = 0;
    this.active = false;
  }

  push(frame, sampleRate) {
    const frameMs = frame.length / sampleRate * 1000;
    const voiced = isSpeech(frame);

    if (voiced) {
      this.active = true;
      this.speechMs += frameMs;
      this.silenceMs = 0;
      return null;
    }

    if (!this.active) return null;
    this.silenceMs += frameMs;

    const threshold = this.speechMs < 1800 ? 900 : 1350;
    if (this.silenceMs >= threshold) {
      return { type: "endpoint", speechMs: this.speechMs };
    }

    return null;
  }
}

```

真正实现时，`isSpeech()` 不要只靠瞬时 RMS。至少综合：

- RMS;
- peak;
- 连续有声帧数量;
- 基频是否存在;
- voiced ratio;
- HNR 或类似的周期性指标。

还要给“已唤醒连续对话”和“待机唤醒”使用不同门槛。待机宁可严格，避免环境声消耗模型；连续对话应该稍宽松，否则一句真实话可能因为 HNR 差 0.1dB 被拒绝，界面却一直停在“我在听”。

— 8. 唤醒词：不要让通用 ASR 同时做人声门和关键词门

短唤醒词是一个很容易掉坑的地方。

如果给 Whisper 一个强 `initial_prompt`，再放宽无声阈值，它可能把键盘声、衣物摩擦声甚至空调声补成唤醒词。把阈值收紧，又会漏掉用户轻声说的短词。

这两个目标本来就不该交给同一个阈值：

- “这是不是持续真人声？”是声学问题；
- “这是不是我的唤醒词？”是匹配问题。

我们最后用了三道门。

8.1 第一道：持续真人声门禁

先提取廉价声学特征：

```
@dataclass
class VoicePresence:
    duration: float
    rms: float
    peak: float
    voiced_ratio: float
    hnr_db: float
    pitch_hz: float | None

def has_sustained_voice(x: VoicePresence) -> bool:
    strict = (
        x.voiced_ratio >= 0.12
        and x.hnr_db >= 3.5
        and x.rms >= 0.004
        and x.peak >= 0.04
    )

    relaxed_short_voice = (
        x.duration <= 3.5
        and x.voiced_ratio >= 0.06
        and x.hnr_db >= 2.0
        and x.rms >= 0.004
        and x.peak >= 0.04
        and x.pitch_hz is not None
    )

    return strict or relaxed_short_voice
```

这些数字不是普适标准，只是合理起点。你需要收集自己设备上的正负样本，再画分布。最重要的原则是：声学分析失败时，待机唤醒应 fail closed。

8.2 第二道：个人 **query-by-example** 匹配

让用户录入 4 次当前唤醒词。每个样本经过本地 speech embedding 模型，保存为个人模板。识别候选同样提取 embedding，再用 cosine distance + DTW 匹配。

```
def wake_similarity(candidate_frames, templates) -> float:
    candidate = embed(candidate_frames)
    return min(cosine_dtw(candidate, template) for template in templates)
```

个人模板要按标准化唤醒词隔离：

```
enrollment/
<sha256(normalized_wake_word)>/
sample-1.npy
sample-2.npy
sample-3.npy
sample-4.npy
```

改了唤醒词，就不能继续误用旧模板。

还要检查“整段是否像一个孤立的唤醒词”。如果候选有效声段明显比录入中位时长长很多，不要允许 matcher 从普通长句中滑出一小段相似片段并唤醒。

8.3 第三道：常驻 **tiny ASR** 文本复核

个人声学近似通过后，才把最可能的 1.6 秒片段交给常驻 tiny Whisper 复核。此时短提示词可以用，因为前两道声学门已经显著缩小了误报面。

```
async def verify_wake(candidate, wake_word):
    if not has_sustained_voice(analyze(candidate)):
        return False

    match = personal_match(candidate)
    if not match.accepted:
        return False

    segment = crop_wake_segment(candidate, match)
    result = await tiny_worker.transcribe(
        segment,
        language=language_for(wake_word),
        prompt=wake_word,
        timeout=1.5,
    )

    return result.confidence >= 0.35 and text_matches(
        result.text,
        wake_word,
    )
```

超过 1.6 秒的候选，优先保留末尾通常比保留开头更安全，因为预录音在前面，唤醒词尾音在后面。如果第一次没命中，可以再用 matcher 定位的最佳片段做一次纯本地复核。

任何一步失败都不应调用主聊天模型。这样环境声和失败唤醒不会消耗 token。

8.4 冷却窗口

至少设置三类冷却：

- 前端刚连接后约 3 秒；
- 会话刚结束后约 3 秒；
- 本地 TTS 或唤醒应答结束后约 260ms。

它们用于避开发设备初始化噪声、播放尾音和回声残留。

9. ASR：远端速度与本地质量并行竞速

只依赖远端 ASR，网络抖动会拖出长尾；只依赖本地大模型，首次加载和硬件差异又很明显。

较好的结构是：

```
远端快速 ASR ┌┐
                │├─ 结果仲裁 ─> 最终文本
常驻本地 ASR └┘
```

9.1 为什么本地模型必须常驻

每轮启动一次 CLI，意味着每轮都重新加载模型。即使热识别只有 600ms，冷启动也可能变成 8 – 15 秒。

常驻 worker 使用严格的 JSON Lines 协议：

```
{"id":"req_7","audio_b64":"...", "language":"zh","task":"transcribe"}
{"id":"req_7","ok":true,"text":"你好","confidence":0.91}
```

父进程负责：

- 单消费者串行化；
- 请求 ID 与取消 ID；
- 最大输入字节数；
- 最大输出行长度；
- 每次请求 deadline；
- worker 崩溃后重建；
- 服务关闭时唤醒阻塞读并回收；
- 空闲一定时间后可选择释放模型。

不要把“子进程还活着”当成“模型已预热”。可以在服务启动和桌面客户端连接时显式调用 `warmup`，再记录首次真实转写是否仍承担加载时间。

9.2 格式标准化

浏览器可能上传 WebM/Opus，而本地 worker 只接受 PCM WAV。远端成功时这个问题不会暴露，一旦远端超时，本地兜底就会立即报 `invalid_audio`，表面看起来像“偶发空回”。

在共享总预算内做格式转换：

```
async def normalize_for_asr(blob: bytes, mime: str, deadline: Deadline) -> bytes:
    if is_pcm16_mono_wav(blob, sample_rate=16_000):
        return blob

    return await run_ffmpeg(
        blob,
        output_rate=16_000,
        channels=1,
        timeout=min(1.0, deadline.remaining()),
    )
```

转换时间必须算在 ASR 总 `deadline` 里，不能悄悄新增一段无限等待。

9.3 竞速不是简单的“谁先返回用谁”

远端通常快，本地大模型通常更准。我们采用“远端先到后给本地一个很短的质量宽限”：


```

async def recognize(audio, total_timeout=7.0, quality_grace=0.9):
    remote_task = asyncio.create_task(remote_asr(audio))
    local_task = asyncio.create_task(local_asr(audio))

    done, _ = await asyncio.wait(
        {remote_task, local_task},
        return_when=asyncio.FIRST_COMPLETED,
        timeout=total_timeout,
    )

    if local_task in done and valid(local_task.result()):
        cancel(remote_task)
        return local_task.result()

    if remote_task in done and valid(remote_task.result()):
        try:
            local = await asyncio.wait_for(local_task, timeout=quality_grace)
            if valid(local):
                return local
        except asyncio.TimeoutError:
            pass
        cancel(local_task)
        return remote_task.result()

    return await bounded_fallback(done, remote_task, local_task)

```

生产实现还要处理：

- 远端空文本；
- 文本长度与音频时长明显不相称；
- 用户已打断或挂断；
- 本地任务取消后底层推理仍短暂运行；
- 结果到达时该 turn 已不再活跃。

取消句柄必须绑定 turn identity。否则旧轮次的“取消本地识别”可能误杀下一轮刚复用的 worker 请求。

9.4 远端重试要考虑是否可能已经送达

网络错误不能一律换线路重发。可安全重试的通常是请求交付前失败，例如连接失败、连接超时、代理握手失败。读超时、写错误、远端协议错误可能意味着音频已经送达，再发一次可能产生重复请求和重复计费。

把错误分成：

```

safe_to_retry_elsewhere
ambiguous_delivery
definitive_response_error

```

`ambiguous_delivery` 应直接进入本地兜底，不要盲目重放远端请求。

10. 语气感知不要阻塞正式回答

音高、语速、停顿、明亮度、气声和稳定度能给聊天模型提供有价值的隐藏上下文，但它们是辅助信息。

曾经有一轮 ASR 只花了约 0.9 秒，后续语气分析却同步等了八秒，用户看到的就是“识别很慢”。后来我们把电话快路径改成：

1. 只做本地声学分析；
2. 生成短结构化摘要；
3. 放进独立线程；
4. 给 250 – 350ms 的硬预算；
5. 超时就省略，不阻塞模型请求。

```
try:
    tone = await asyncio.wait_for(
        asyncio.to_thread(analyze_tone, wav),
        timeout=0.25,
    )
except asyncio.TimeoutError:
    tone = None
```

语气基线若需要持续更新，要用进程内锁和跨进程文件锁，并以临时文件 + `fsync` + `os.replace` 原子写入。更重要的是：只有当前 turn 的分析正常完成后才更新基线。已取消轮次的后台线程即使后来结束，也不能污染长期统计。

11. 模型层：保留当前模型，但给语音一套轻上下文

为了低延迟，很多系统会把语音请求强制改到某个小模型。这样做简单，却破坏了一个重要预期：用户在主聊天里选了哪个模型，语音也应该是同一个“人”。

我们把两个问题拆开：

- 用哪个模型；
- 给模型多少上下文和工具。

普通语音闲聊继续使用会话当前模型，但走轻量执行计划：

```

@dataclass
class VoiceExecutionPlan:
    requested_model: str
    effective_model: str
    use_compact_context: bool
    include_tool_schema: bool
    allow_full_memory_retrieval: bool

def plan_voice_turn(text, requested_model, attachments):
    complex_intent = detects_tool_or_complex_intent(text, attachments)

    return VoiceExecutionPlan(
        requested_model=requested_model,
        effective_model=requested_model,
        use_compact_context=not complex_intent,
        include_tool_schema=complex_intent,
        allow_full_memory_retrieval=complex_intent,
    )

```

轻量上下文保留：

- 人格与关系规则；
- 用户基础信息；
- 最近几轮对话；
- 当前时间和必要状态；
- 语音输出格式规则；
- 明确的长期记忆线索。

普通闲聊可以省略：

- 几百个用不到的工具 schema；
- 重型向量检索；
- 动态能力说明全集；
- 与当轮无关的大块状态；
- 后台摘要生成。

涉及搜索、文件、记忆、媒体控制或复杂任务时，再切回完整工具路径。这样优化的是上下文成本，不是把语音变成“低配人格”。

11.1 快候选失败后必须回到稳定主模型

如果你仍想先试一个低延迟候选，不要让候选超时直接变成“请再说一次”。用户的音频已经成功识别和保存，失败的是模型启动，不该要求用户重说。

```

async def stream_voice_reply(plan, messages):
    for candidate in plan.fast_candidates:
        result = await stream_until_first_text(
            candidate,
            messages,
            timeout=4.0,
        )
        if result.started:
            return result.stream

    # 稳定终端候选不再套激进首字超时
    return await stream_model(plan.requested_model, messages)

```

一旦某个候选已经对外输出可见文字，就锁定它。不能在半句话后悄悄换模型。

11.2 给模型明确的“可说首句”合同

普通文本模型经常先输出长铺垫，或在工具请求里沉默很久。语音提示应明确要求：

- 先给一条 4 - 14 个汉字左右、可独立成立的真实短句；
- 短句必须贴合用户刚说的内容；
- 查看或工具请求可以先说明真实动作；
- 不能编造尚未得到的结果；
- 后续再展开完整回答。

例如：

```

用户：帮我找一下附近还开着的面馆。
首句：好，我看看附近还营业的。
后续：工具执行后再给真实结果。

```

这比固定播放“让我想想”更像对话，因为第一声已经是模型对当下内容的真实反应。

12. TTS：首句优先切分 + 持久连接 + 有序播放

12.1 不要等完整答案

TTS streamer 一边接收模型文本，一边找自然切点：

```

class TTSSreamer:
    def __init__(self):
        self.buffer = ""
        self.seq = 0

    async def feed(self, delta: str):
        self.buffer += delta

        while cut := find_cut_position(
            self.buffer,
            first_min=4,
            soft_min=8,
            hard_min=14,
        ):
            text = self.buffer[:cut]
            self.buffer = self.buffer[cut:]
            await self.emit(text, self.seq)
            self.seq += 1

```

切分优先级大致是：

1. 句号、问号、感叹号；
2. 逗号、分号等自然停顿；
3. 达到硬长度后的安全字符边界；
4. 避免把数字、英文单词、URL、Markdown 标记切坏。

首段可以更激进，后续段稍长，减少碎裂感。

12.2 复用 WebSocket TTS 会话

每个分段都新建 HTTP 客户端会浪费连接、TLS 和服务端会话准备时间。支持 WebSocket 流式 TTS 时，可以按这些维度建立连接池：

事件循环 + TTS 模型 + voice ID + 音频格式 + 采样设置

```

class TTSTConnectionPool:
    async def acquire(self, key):
        session = self.sessions.get(key)
        if session and session.healthy:
            return session

        session = await MiniMaxTTSWebSocketSession.connect(key)
        self.sessions[key] = session
        return session

```

创建 streamer 时就预热首选模型。WebSocket 失败后，可以回到同模型 HTTP，再回退到备用高质量模型。回退必须保持同一个 seq，客户端不需要知道底层换了传输方式。

12.3 OrderedTTSQueue

TTS 片段可能乱序返回：seq=1 先到，seq=0 后到。播放器必须只按序消费。

```
class OrderedTTSQueue {
  constructor(identity) {
    this.identity = identity;
    this.nextSeq = 0;
    this.pending = new Map();
    this.cancelled = false;
  }

  push(chunk) {
    if (this.cancelled) return;
    if (!belongsTo(chunk, this.identity)) return;
    this.pending.set(chunk.seq, chunk);
    this.drain();
  }

  async drain() {
    while (!this.cancelled && this.pending.has(this.nextSeq)) {
      const chunk = this.pending.get(this.nextSeq);
      this.pending.delete(this.nextSeq);
      await playChunk(chunk);
      this.nextSeq += 1;
    }
  }

  cancel() {
    this.cancelled = true;
    this.pending.clear();
    stopCurrentAudio();
  }
}
```

预加载音频对象只能 load()，不能偷偷 play()。我们曾经见过第二段预加载对象抢先播放，造成重叠。真正的播放权只能由队列头拥有。

TTS 事件最好同时携带该分段的清洗后文本。客户端在 audio.play() 前刷新字幕或桌宠气泡，文字与声音会从第一段开始同步，不必等最终消息落库。

13. 本地短反馈：填空档，但不能抢真实回答

本地短反馈的价值很大，风险也很大。用得不好，每轮都是机械“嗯”“让我想想”，比沉默更烦。

我们把反馈分成几类：

address_ack	用户说话中短称呼后的轻回应
opening_ack	普通开场或短停顿回应
test_lead_in	“试一下延迟”这类测试语境
search_lead_in	明确查、搜、找
thinking_lead_in	需要判断、比较、反思的问题
sharing_ack	用户分享自己的事
fun_ack	明显的趣事或玩笑
neutral_lead_in	其他成功转写的轻量兜底

分类应基于最终转写，而不是单纯在 endpoint 后 200ms 就播放。否则“我先挂了”“你能听到吗”也会先冒出一个不合时宜的“嗯”。

```
def classify_lead_in(text: str) -> str:
    if is_call_state_question(text):
        return "instant_local_reply"
    if is_farewell(text):
        return "none"
    if is_explicit_search(text):
        return "search_lead_in"
    if is_test_context(text):
        return "test_lead_in"
    if is_reflective_question(text):
        return "thinking_lead_in"
    if is_personal_sharing(text):
        return "sharing_ack"
    return "neutral_lead_in"
```

13.1 素材选择器

每个类别准备 3 条以上短素材，并记录：

- clip_id ;
- semantic_class ;
- tone_family ;
- playback_volume ;
- cooldown_seconds ;
- complete_before_formal ;
- expression_family 。

```
{
  "id": "thinking_03",
  "semantic_class": "thinking_lead_in",
  "tone_family": "soft-thinking",
  "playback_volume": 0.52,
  "cooldown_seconds": 90,
  "complete_before_formal": false,
  "expression_family": "thinking-short"
}
```

选择规则可以是：

- 最近四条不重复；
- 连续同语调降权；
- 显眼句式有更长冷却；
- 同一表达家族跨类别共享冷却；
- 低频亲昵岔话只能在多轮、长会话后小概率出现；
- 喝水、休息等主动提醒要有更长冷却。

13.2 正式首句拥有最高优先级

普通短反馈正在播时，正式 TTS seq=0 到达，应立即停止短反馈并接管。不要为了播完一句预设录音，延迟模型的真实回答。

只有极少数明确标记 `complete_before_formal=true` 的长反馈可以先播完，但它们应该低频出现。

```
if (formalSeq0Ready && !leadIn.completeBeforeFormal) {
  stopLeadIn();
  await playFormalSeq0();
}
```

即使短反馈被中途打断，也要把它计入最近播放和冷却，否则下一轮它可能立刻再次被选中。

13.3 用户仍在说话时的监听回应

如果想做“宝宝……你能听到吗”中间那种轻声回应，不必完全依赖 ASR 把短称呼认对。可以使用声学节奏：

```
短开场发声
→ 自然微停顿
→ 同一轮在 900ms 内继续说话
→ 播一次短监听回应
```

只有“继续说话”发生后才播放。若用户停顿后没有继续，就把它当作一句可能已经结束的话，避免仅凭停顿乱回应。

这条路径完全在客户端工作，不等待远端 ASR。它仍然要绑定 call/turn/request identity，并在新轮次或挂断时取消。

■ 14. 真正的核心：自然抢话（barge-in）

“AI 播放时麦克风还开着”不等于实现了抢话。扬声器回声、短促咳嗽、桌面碰撞都可能误触。

我们使用两阶段检测：

约 240ms 连续人声 → duck：先把 TTS 音量压低
约 520ms 连续人声 → interrupt：确认用户真的要抢话
短促误触消失约 160ms → restore：恢复原音量

同时保留约 1000ms 的 PCM 环形预卷，确认打断后把用户开头灌回正式分段器。

```

class BargeInDetector {
  constructor({ duckMs = 240, interruptMs = 520, restoreMs = 160,
               prerollMs = 1000 }) {
    this.duckMs = duckMs;
    this.interruptMs = interruptMs;
    this.restoreMs = restoreMs;
    this.preroll = new PcmRingBuffer(prerollMs);
    this.voicedMs = 0;
    this.silentMs = 0;
    this.ducked = false;
  }

  push(frame, sampleRate) {
    this.preroll.push(frame, sampleRate);
    const ms = frame.length / sampleRate * 1000;

    if (isLikelyUserSpeech(frame)) {
      this.voicedMs += ms;
      this.silentMs = 0;

      if (!this.ducked && this.voicedMs >= this.duckMs) {
        this.ducked = true;
        duckCurrentPlayback();
      }

      if (this.voicedMs >= this.interruptMs) {
        return {
          type: "interrupt",
          preroll: this.preroll.snapshot()
        };
      }
    } else {
      this.silentMs += ms;
      if (this.ducked && this.silentMs >= this.restoreMs) {
        restoreCurrentPlayback();
        this.resetEvidence();
      }
    }

    return null;
  }
}

```

确认打断后，动作顺序很重要：

1. 生成新的 turn identity;

2. 将旧 generation 标为 cancelled;
3. 停止当前正式 TTS;
4. 停止本地短反馈和思考前导;
5. 清空旧 TTS pending chunks;
6. 通知后端取消旧模型流和未完成 TTS;
7. UI 回到 listening;
8. 把预卷 PCM 喂入新一轮录音器;
9. 继续接收用户后续语音。

后端中断接口必须校验:

```
client_id + call_session_id + generation_id
```

```
async def interrupt_turn(request):
    session = sessions.get(request.call_session_id)
    if not session:
        return {"ok": False, "reason": "unknown_call"}

    if session.active_generation_id != request.generation_id:
        return {"ok": False, "reason": "stale_generation"}

    await cancel_chat_generation(request.generation_id)
    await cancel_tts_generation(request.generation_id)
    session.interrupt_current_turn()
    session.mode = "listening"
    return {"ok": True}
```

“点击关闭”与“自然抢话”不要共用语义:

- 自然抢话只结束当前回复，连续会话还在;
- 点击关闭结束整场会话，停止监听并释放设备。

15. 回声控制：避免把 AI 自己当成用户

扬声器播放期间持续监听，必然要处理回声。

几个实用策略:

15.1 播放阶段临时启用 AEC

如果平台允许，可以在 AI 播放 TTS、问候或本地反馈时使用 `echoCancellation`。正常用户录音和原声归档仍保留高保真配置。

15.2 给本地播放维护时间窗

记录每段播放的开始、结束、音量和约 220ms 尾窗。声学检测发生在这段窗口内时提高判定门槛，但不要直接把所有声音丢掉，因为用户可能真的在同时说话。

15.3 反馈音不能触发全局熔断

监听短音与用户继续说话经常重叠，这是预期。不要把一次 overlap 当成“回声污染”，然后关闭整通电话的监听回应。

15.4 手机端使用 voice-chat 音频会话

原生客户端应使用适合通话的录制模式，让系统级 AEC 有机会工作。Web、Electron、iOS 的音频合同要分别测试，不能因为浏览器通过就推断手机也通过。

16. 自然挂断

用户说“我先挂了”时，立即断开会令 AI 的告别被截断；一味等待又可能被旧任务卡死。

使用 identity-bound graceful hangup:

1. ASR 将这一轮分类为 farewell;
2. 生成告别回答;
3. 等待匹配 generation 的模型流完成;
4. 等待同 generation 的 TTS 队列真正排空;
5. 再结束 call;
6. 整个过程有硬截止时间。

```
async function replyThenHangup(identity) {
  if (!identity?.callId || !identity?.generationId) {
    return { ok: false, reason: "voice_identity_unavailable" };
  }

  await waitForGenerationStream(identity, { timeoutMs: 30_000 });
  await waitForTtsQueueEmpty(identity, { timeoutMs: 30_000 });

  if (!isStillActive(identity)) {
    return { ok: false, reason: "stale_generation" };
  }

  return endCall(identity.callId);
}
```

注意显式 `null`。JavaScript 的默认参数只能兜住 `undefined`，不能阻止 `null.callId` 抛异常。所有身份入口都用可选链或显式 schema 校验。

旧轮次播放完成、自动播放重试、空 TTS、网络失败和 deadline 都不能结束一通更新的电话。

17. 一轮完整对话的真实时序

下面是一轮普通问题：

普通一轮：本地短反馈填空档，真实首句立刻抢占播放权

- | | |
|--------|------------------------|
| 01 用户 | 说话，客户端用 VAD 判断停句。 |
| 02 ASR | 远端与本地并行，输出最终文字和反馈分类。 |
| 03 播放器 | 先播放内容相关的本地短反馈，避免空白等待。 |
| 04 模型 | 使用轻上下文流式输出第一段可说文本。 |
| 05 TTS | 首句优先合成，返回 seq=0 的正式音频。 |
| 06 仲裁器 | 中断短反馈，严格按序播放真实回答。 |

用户在 AI 播放时抢话：

自然抢话：先 **duck**，再确认 **interrupt**

240ms

检测到真人开口，压低当前 AI 播放，保留预卷。

520ms

确认抢话，向后端发送 interrupt 并取消旧生成。

next turn

清空旧 TTS 队列，进入 listening，接住用户新句。

旧模型任务、旧 TTS、旧播放 Promise 都只能操作自己的 generation。

18. 推荐项目结构

不必照抄文件名，但边界最好清楚：

```

voice-system/
├── backend/
│   ├── voice_protocol.py      # call/turn/generation 协议
│   ├── voice_session.py      # 状态机与取消
│   ├── voice_presence.py     # 真人声声学门
│   ├── wake_matcher.py       # 个人模板与 DTW
│   ├── prefix_recognizer.py   # tiny 常驻 worker manager
│   ├── final_recognizer.py    # 最终 ASR worker manager
│   ├── asr_router.py         # 远端/本地竞速
│   ├── voice_intent.py       # 反馈、挂断、工具意图分类
│   ├── voice_latency.py      # 分阶段指标
│   ├── tts_streamer.py       # 文本切分
│   ├── tts_websocket.py      # 持久 TTS 会话和连接池
│   └── routes_voice.py       # HTTP/WebSocket 路由
├── frontend/
│   ├── pcm-worklet.js        # 音频线程采集
│   ├── endpoint-detector.js  # 自适应停句
│   ├── barge-in-detector.js  # 两阶段打断与预卷
│   ├── ordered-tts-queue.js  # 有序播放
│   ├── listener-cues.js      # 本地反馈选择
│   ├── call-runtime.js       # 会话状态机
│   └── voice-ui.js           # 单一 UI 渲染入口
├── assets/
│   ├── voice-cues/
│   │   ├── manifest.json
│   │   └── *.mp3
├── workers/
│   ├── prefix_worker.py
│   └── final_asr_worker.py
└── tests/
    ├── test_protocol.py
    ├── test_endpoint.js
    ├── test_barge_in.js
    ├── test_asr_race.py
    ├── test_tts_queue.js
    └── test_voice_integration.py

```

每个模块只回答一个问题。比如 `BargeInDetector` 只判断“是否该 duck/interrupt”，不直接发网络请求；`OrderedTTSQueue` 只处理身份、顺序和播放，不决定文本该如何切分。

19. 一组可落地的事件协议

19.1 客户端到服务端

```
{
  "event": "voice_turn_audio",
  "protocol_version": 1,
  "call_session_id": "call_01",
  "turn_id": "turn_03",
  "turn_sequence": 3,
  "request_sequence": 3,
  "audio_format": "wav_pcm16_mono_16000",
  "archive_format": "audio/webm; codecs=opus"
}
```

```
{
  "event": "voice_interrupt",
  "client_id": "desktop-client",
  "call_session_id": "call_01",
  "generation_id": "gen_03_a"
}
```

19.2 服务端到客户端

```
{
  "event": "voice_state",
  "mode": "thinking",
  "call_session_id": "call_01",
  "turn_id": "turn_03",
  "generation_id": "gen_03_a",
  "message": ""
}
```

```
{
  "event": "tts_chunk",
  "call_session_id": "call_01",
  "turn_id": "turn_03",
  "generation_id": "gen_03_a",
  "seq": 0,
  "text": "我先看看这个。",
  "audio_url": "/voice/audio/chunk-0.mp3"
}
```

```
{
  "event": "voice_metrics",
  "call_session_id": "call_01",
  "turn_id": "turn_03",
  "generation_id": "gen_03_a",
  "asr_ms": 812,
  "model_first_text_ms": 2860,
  "tts_first_audio_ms": 604
}
```

协议里所有 ID 都由明确一方生成，并在一轮中保持不变。不要让 TTS 自己再偷偷生成一个与聊天 generation 无关的 ID。

— 20. 环境配置示例

下面只列功能开关，不包含任何真实密钥：


```
VOICE_NATURAL_ENABLED=1
VOICE_ADAPTIVE_ENDPOINT_ENABLED=1
VOICE_BARGE_IN_ENABLED=1
VOICE_LISTENER_CUES_ENABLED=1
VOICE_CALL_GREETING_ENABLED=1
VOICE_FAST_ASR_ENABLED=1
VOICE_FINAL_LOCAL_WORKER_ENABLED=1
VOICE_FINAL_LOCAL_BACKEND=mlx-whisper
VOICE_FINAL_LOCAL_MODEL=/path/to/local/whisper-model
VOICE_FINAL_ASR_DEADLINE_MS=7000
VOICE_PREFIX_MODEL=tiny
VOICE_PREFIX_TIMEOUT_MS=1500
VOICE_TONE_BUDGET_MS=250
VOICE_MODEL_FIRST_TEXT_TIMEOUT_MS=4000
VOICE_STREAMING_TTS_ENABLED=1
VOICE_GENERIC_ENDPOINT_CUE_ENABLED=0
```

功能开关很有用。生产出了问题，可以只关闭某个子系统：

- 关闭 listener cues，不影响正式回答；
- 关闭本地 ASR worker，仍可用远端 ASR；
- 关闭快速模型尝试，回到稳定主模型；
- 关闭 barge-in，不必回滚整套语音协议；
- 关闭个性化唤醒，仍可保留手动麦克风。

总开关关闭时，所有子功能都应 fail closed，而不是留下半套运行时。

21. 分阶段实现，不要一次把所有异步都打开

推荐按下面顺序升级。

阶段 A：先修好身份和顺序

实现：

- call/turn/generation 协议；
- 状态机；
- OrderedTTSQueue；
- 旧 generation 丢弃；
- 后端取消接口。

验收：用户快速连续发两轮，第一轮任何迟到片段都不能出声。

阶段 B：降低正式首声延迟

实现：

- 模型流式输出；
- 首句优先文本切分；
- TTS 持久连接；
- TTS 首段预热；
- 分阶段 latency metrics。

验收：可以明确看出瓶颈在 ASR、模型还是 TTS。

阶段 C：加入本地反馈与监听回应

实现：

- 内容分类；
- 本地素材 manifest；
- 冷却与去重；
- 正式 seq=0 抢占反馈；
- 用户说话中的声学节奏回应。

验收：普通陈述不再每轮机械“嗯”；挂断和通话状态问题不播放错误反馈。

阶段 D：自然抢话

实现：

- 播放期间持续监听；
- 两阶段 duck/interrupt；
- PCM 环形预卷；
- 端到端 generation 取消；
- 回声尾窗。

验收：在问候、本地反馈、思考前导、正式第 0 段和后续段分别抢话，开头都不能被吞。

阶段 E：个性化唤醒和连续会话

实现：

- 真人声门；
- 4 次个人录入；
- embedding + DTW；
- tiny 文本复核；
- 常驻最终 ASR；
- 待机/连续对话不同门槛。

验收：键盘声、播放回声和普通长句不唤醒；真实轻声唤醒有可接受召回率。

阶段 F：原声归档与跨端一致性

实现：

- AudioWorklet 分析流；

- MediaRecorder 归档流；
- WebM/WAV 后端格式兜底；
- Web、Electron、iOS 各自的音频会话设置；
- 单一 UI 状态渲染。

验收：回放速度、音高、停顿正确；切后台、重连和页面热更新后状态不漂移。

— 22. 测试：这类系统必须测“时序”，不只是测返回值

22.1 单元测试

至少覆盖：

- 短句和长句 endpoint；
- 240ms duck、520ms interrupt、160ms restore；
- 预卷包含打断开头；
- TTS seq=1 先到时不能抢播；
- cancelled generation 永久拒绝迟到片段；
- null identity 不抛异常；
- 本地短反馈被正式首段抢占；
- 被抢占反馈仍进入冷却；
- 旧 audio.play() Promise 不能清理新播放；
- farewell 必须等待正确 generation 排空；
- ASR 转换超时仍受总 deadline 限制；
- 远端读超时不跨线路重复提交；
- 本地 worker 崩溃后能重建；
- 唤醒词个人模板按词隔离。

22.2 用虚拟时钟测时序

Node 测试可以用 fake timers：

```
detector.push(voicedFrameFor(239));
assert.equal(audio.ducked, false);

detector.push(voicedFrameFor(1));
assert.equal(audio.ducked, true);

detector.push(voicedFrameFor(280));
assert.equal(interrupts.length, 1);
```

不要在单元测试里真的 `sleep(520ms)`，否则套件又慢又不稳定。

22.3 录音回归集

保存脱敏后的本地样本，不上传：

```
positive/  
  light-wake-word.wav  
  fast-mixed-language.wav  
  long-reflective-question.wav  
negative/  
  keyboard.wav  
  clothes-rustle.wav  
  speaker-echo.wav  
  normal-sentence-containing-similar-syllable.wav
```

每次调门槛，同时跑正负样本。只拿一条真实唤醒调到成功，很容易把环境噪声也一起放进来。

22.4 竞态测试

主动制造乱序：

- seq 2、0、1 顺序返回；
- 旧轮 fetch 在新轮开始后 resolve；
- `audio.play()` 在挂断后 reject；
- WebSocket 重连时旧状态补发；
- 页面初始化状态晚于实时推送；
- ASR 刚完成时用户挂断；
- TTS 队列排空与新 call 同时发生。

22.5 真实设备验收

自动化证明不了听感。至少在真实麦克风和扬声器上连续测试十轮：

1. 短句快速停顿；
2. 长句中间思考停顿；
3. 轻声唤醒；
4. 背景音乐下唤醒；
5. AI 问候中抢话；
6. 本地反馈中抢话；
7. 正式首段中抢话；
8. 工具请求；
9. 通话状态问题；
10. 自然告别挂断。

每轮同时看结构化日志，不能只凭“感觉有点慢”猜是哪段坏了。

23. 我们踩过的坑，以及它们真正说明了什么

坑 1：靠 **ASR** 提示词做人声检测

结果：键盘声被补成唤醒词。

教训：先做独立声学门，再做关键词识别。提示词只能是门后的辅助证据。

坑 2：本地 **Whisper** 每轮冷启动

结果：平时几百毫秒，偶发十几秒。

教训：最终 ASR 和前缀 ASR 都要常驻 worker，并在客户端连接时预热。

坑 3：远端超时后把 **WebM** 原样送给 **WAV-only worker**

结果：远端一抖，本地兜底也立刻失败，界面只显示“没听清”。

教训：格式兼容要在失败路径上专门测试，日志必须记录 backend、ok 和 error category。

坑 4：主线程 **PCM** 既做分析又做回放归档

结果：主线程漏帧后回放像加速。

教训：AudioWorklet 负责实时分析；带时间戳的 MediaRecorder 负责归档。

坑 5：**endpoint** 一到就机械播放“嗯”

结果：挂断、状态确认、普通陈述都被抢话。

教训：通用反馈至少等最终 ASR 分类；说话中的监听回应则走独立声学节奏路径。

坑 6：为了低延迟强制换成固定小模型

结果：语音和主聊天像两个不同的人，工具能力也缩水。

教训：模型选择和上下文轻量化是两个维度。保留当前模型，只精简普通语音轮的上下文。

坑 7：所有快候选失败就要求用户重说

结果：音频和文字已经成功保存，用户却以为发送失败。

教训：快候选只是优化；最终必须回到稳定的请求模型，且不能重复保存用户消息。

坑 8：语气分析放在关键路径

结果：ASR 很快，辅助模型却拖住正式回答。

教训：辅助感知有独立预算，超时省略。先让对话发生。

坑 9：只有一个 **isSpeaking**

结果：旧 TTS、旧 Promise 和旧 timer 串到下一轮。

教训：call/turn/generation/seq 缺一不可；异步结果先验身份，再改状态。

坑 10：服务重启后以为前端脚本也更新了

结果：已打开的 WebView 继续跑旧 JavaScript，新接口一次也没调用。

教训：服务端状态返回 runtime version；拨号前核对客户端版本；每个目标版本最多自动刷新一次，防止死循环。

坑 11：多个 UI 入口各写各的

结果：实时状态先显示正确，稍晚的 HTTP 返回又把它覆盖成旧状态。

教训：状态渲染必须只有一个入口，轮询只负责把后端真值送进这个入口。

坑 12：把自动化通过当成听感通过

结果：状态机没有错，但音量、回声、停顿或短反馈频率仍不自然。

教训：语音产品必须有物理验收。波形正确不等于人耳舒服。

24. 可观测性与隐私

24.1 记录什么

建议记录：

- call/turn/generation/request sequence；
- 状态迁移；
- 音频时长、字节数、格式；
- 声学指标；
- ASR backend、耗时、错误类别、字符数；
- 模型候选、首字耗时；
- TTS transport、模型、首音频耗时；
- 本地反馈 clip ID、分类、音量；
- duck、interrupt、restore；
- 取消原因和迟到事件丢弃原因。

24.2 不记录什么

默认不要记录：

- 原始音频内容；
- 完整私人转写；
- API key；
- 用户个人唤醒模板；
- 可恢复的账号凭据；
- 未脱敏的长期记忆内容。

调试日志里的转写最多保留很短片段，并允许一键关闭。个人唤醒模板留在本机；删除或更改唤醒词时提供明确清理入口。

24.3 本地模型也要设边界

本地不等于无限可信。worker 输入长度、输出长度、并发、路径和模型目录都要验证。不要让客户端传任意文件路径给 ffmpeg 或 Whisper。

25. 一个最小可用版和一个完整升级版

如果朋友只想先做出 70% 的体验，可以实现下面这套最小版：

最小可用版

- AudioWorklet PCM;
- 自适应 endpoint;
- 整段 ASR;
- 当前模型流式文本;
- 首句优先 TTS;
- OrderedTTSQueue;
- call/turn/generation 身份;
- 240/520ms 两阶段打断;
- 约 1 秒预卷;
- 分阶段延迟日志。

它已经会比传统“录完、等完、播完”自然很多。

完整升级版再增加：

- 个人唤醒样本;
- 真人声门与 tiny 复核;
- 远端/本地 ASR 并行竞速;
- 常驻 MLX Whisper;
- 内容感知本地反馈;
- 声学节奏监听回应;
- 持久 WebSocket TTS;
- 原声归档双轨;
- 自然挂断;
- 跨端音频会话;
- runtime version 自愈;
- UI 状态对账。

建议先把最小版的身份、取消和播放顺序做扎实，再加“更聪明”的感知功能。否则每增加一条并行链路，竞态都会成倍增加。

26. 上线前检查表

身份与取消

- ☐ 每个事件都有 call、turn、generation 身份
- ☐ TTS chunk 有严格递增 seq
- ☐ 旧 generation 的迟到结果永久丢弃
- ☐ 用户打断会同时取消模型、TTS 和本地反馈
- ☐ 旧 Promise/timer 不会修改新轮状态

音频

- ☐ 实时分析使用 AudioWorklet
- ☐ ASR 输入格式固定或有后端转换兜底
- ☐ 回放归档不复用低采样率识别副本
- ☐ 录音时长、WAV 头和实际播放时长一致
- ☐ Web、Electron、iOS 分别做过物理测试

识别

- ☐ 待机唤醒前有真人声门
- ☐ 个人模板与唤醒词隔离
- ☐ 本地模型常驻并已预热
- ☐ ASR 总 deadline 包含上传、转换和兜底
- ☐ 远端不安全错误不会跨线路重复提交

对话与 TTS

- ☐ 普通语音使用当前会话模型
- ☐ 工具意图能切回完整上下文
- ☐ 模型先输出可说的真实短句
- ☐ TTS 首段优先切分
- ☐ WebSocket 失败有 HTTP 和备用模型兜底
- ☐ 正式 seq 0 可以抢占普通本地反馈

双工体验

- ☐ 240ms duck 与 520ms interrupt 已调过听感
- ☐ 打断时不会吞首字
- ☐ 扬声器回声不会持续误触
- ☐ 用户继续说话时监听回应不会终止录音
- ☐ 点击关闭与自然抢话语义不同
- ☐ 告别在正确 generation 播完后才挂断

运维

- ☐ 每阶段都有结构化耗时
- ☐ 日志不保存原始音频和凭据
- ☐ 子功能可以独立关闭
- ☐ runtime version 不一致会安全刷新
- ☐ 页面状态、WebSocket 和后端状态能自动对账

27. 结语

做完这套系统后，我最大的感受是：自然语音并不主要取决于“模型能不能说话”，而取决于系统是否尊重对话的时间。

用户什么时候开始说、什么时候只是停顿、什么时候真的想打断；模型的第一句什么时候已经足够真实，可以先说出来；一个旧任务什么时候已经失去发言权；本地反馈什么时候该填空，什么时候必须闭嘴。这些看起来是小细节，凑在一起才形成“像在和一个人通话”的感觉。

如果只记住一条原则，就记住这一条：

让每条链路并行工作，让每个结果带着身份回来，让真实回答永远拥有最高优先级。

这样即使底层仍是 ASR、文本模型和 TTS，也能做出非常接近实时双工语音的自然体验；以后换成增量 ASR、WebRTC 或原生音频到音频模型时，上层状态机、取消协议、播放仲裁和测试体系也不需要推倒重来。